WEB APPLICATION SECURITY TESTING

Security Assessment of a Deliberately Vulnerable Web Application

Submitted By

SHUBHAM RAJENDRA DUBELE

Level: Intermediate / Level 2

Laboratory Environment: Kali Linux + DVWA

Tools: Nmap • Nikto • Burp Suite • Firefox • DVWA

College Cybersecurity Project Report

1. Abstract

This project presents a controlled web application security assessment performed against a deliberately vulnerable application in an isolated laboratory environment. Kali Linux was used as the security-testing platform, while Damn Vulnerable Web Application (DVWA) was used as the target. Nmap, Nikto, and Burp Suite were used for service discovery, web-server assessment, HTTP traffic interception, and application-level testing. SQL Injection and Cross-Site Scripting (XSS) were demonstrated within the lab to understand their behavior, impact, and appropriate mitigations.

2. Introduction

Modern web applications process user input, authenticate users, access databases, and expose functionality through HTTP. Weak input handling, unsafe output processing, and insecure server configuration can introduce vulnerabilities. Security testing helps identify these weaknesses so that developers and administrators can apply appropriate controls before deployment.

3. Problem Statement

To identify and analyze common web application security weaknesses in a controlled, intentionally vulnerable environment and document their potential impact together with practical mitigation strategies.

4. Objectives

• Set up an isolated web-security testing laboratory.
• Perform basic target and service discovery using Nmap.
• Assess the web server using Nikto.
• Intercept and analyze HTTP requests using Burp Suite.
• Demonstrate SQL Injection in the DVWA laboratory module.
• Demonstrate Cross-Site Scripting (XSS) in the DVWA laboratory module.
• Analyze findings and assign appropriate risk levels.
• Recommend security controls and mitigation techniques.

5. Scope

Testing was restricted to an intentionally vulnerable DVWA instance running in an authorized local laboratory environment. No real-world websites, accounts, third-party systems, or unauthorized networks were included in the assessment.

6. Lab Environment

The recommended architecture uses an isolated Host-Only or Internal Network in VirtualBox/VMware. Kali Linux acts as the testing workstation and DVWA acts as the vulnerable target.

Component	Example Configuration	Purpose
Kali Linux	192.168.56.10	Security testing
DVWA	192.168.56.20:80	Intentionally vulnerable target
Network	Host-Only / Internal	Isolation from normal network

7. Tools Used

Tool	Purpose
Kali Linux	Security testing operating system
DVWA	Deliberately vulnerable web application
Nmap	Port and service discovery
Nikto	Web-server security assessment
Burp Suite	HTTP interception and request analysis
Firefox	Browser-based application testing
VirtualBox / VMware	Virtualized laboratory environment

8. Methodology

The assessment followed a structured workflow: reconnaissance → service discovery → web-server assessment → HTTP interception → vulnerability testing → evidence collection → risk analysis → mitigation recommendations.

9. Application Setup

DVWA may be deployed locally using Docker or a dedicated lab VM. A typical Docker deployment is:
`docker run --rm -it -p 80:80 vulnerables/web-dvwa`
The application is then accessed through the local browser using the appropriate lab address, such as http://127.0.0.1/ or the isolated target VM address.

10. Nmap Service Discovery

Nmap was used to identify reachable services on the authorized laboratory target. Service detection can be performed with:
`nmap -sV 192.168.56.20`
Output can be saved using:
`nmap -sV 192.168.56.20 -oN scans/nmap_service_scan.txt`

11. Nikto Web-Server Assessment

Nikto was used to assess the authorized laboratory web server for common server information, potentially risky files, configuration weaknesses, and security-header observations.
`nikto -h http://192.168.56.20/`
`nikto -h http://192.168.56.20/ -output scans/nikto_report.txt`

12. Burp Suite HTTP Interception

Burp Suite Proxy was configured as the local browser proxy, typically at 127.0.0.1:8080. With interception enabled, browser requests to the DVWA laboratory target can be observed and forwarded. Burp Repeater can then be used to resend and inspect authorized test requests.

13. SQL Injection Testing

The DVWA SQL Injection module was tested using normal input followed by a controlled SQL Injection test payload. A common laboratory demonstration payload is:
`' OR '1'='1`
The purpose is to determine whether user-controlled input is incorporated into a database query without

appropriate parameterization.

14. SQL Injection Impact

Depending on application design, SQL Injection can affect confidentiality and integrity by allowing unintended database queries, unauthorized data access, or data modification. The exact impact depends on database privileges and application behavior.

15. SQL Injection Mitigation

Recommended controls include parameterized/prepared statements, strict input validation, least-privilege database accounts, safe error handling, and secure database configuration.

16. Cross-Site Scripting (XSS) Testing

The DVWA XSS module was tested using a simple controlled JavaScript payload:
`<script>alert('XSS')</script>`
If the laboratory application executes the script rather than treating the content as ordinary text, the test demonstrates XSS behavior.

17. Types of XSS

Reflected XSS: attacker-controlled input is returned by the server and interpreted by the browser.
Stored XSS: attacker-controlled content is stored by the application and later served to users.
DOM-based XSS: client-side JavaScript processes attacker-controlled data in an unsafe way.

18. XSS Impact

Depending on context, XSS can allow manipulation of page content, phishing within a trusted origin, unauthorized actions in a victim's session, or exposure of browser-side information.

19. XSS Mitigation

Recommended controls include context-aware output encoding, input validation, Content Security Policy (CSP), secure cookie attributes, safe DOM APIs, and appropriate sanitization where HTML input is genuinely required.

20. Security Header Analysis

HTTP responses were reviewed for browser security controls such as Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, and Referrer-Policy. Missing headers should be assessed according to application requirements and deployment context.

21. Vulnerability Findings

ID	Finding	Suggested Severity	Evidence
WEB-01	SQL Injection	High	DVWA SQL Injection test
WEB-02	Reflected XSS	Medium	DVWA XSS test
WEB-03	Security-header weakness	Low/Medium	HTTP/Nikto observations
WEB-04	Information disclosure	Low	HTTP/server response

Severity should be validated against the actual application's exposure, exploitability, business impact, and environment.

22. Risk Assessment

A simple qualitative model is: Risk = Likelihood × Impact. High likelihood combined with high impact may justify a Critical/High rating, while low likelihood and low impact generally result in Low risk.

23. Mitigation Summary

Finding	Primary Mitigation
SQL Injection	Prepared statements; input validation; least-privilege DB access
XSS	Context-aware output encoding; CSP; safe DOM handling
Header weakness	Configure appropriate security headers
Information disclosure	Minimize server details and sensitive error messages

24. Evidence / Screenshot Log

Insert your actual screenshots in the sections below. Redact passwords, session cookies, tokens, API keys, and other secrets before sharing the report.

Screenshot 1 — Kali Linux showing IP address

Screenshot 2 — DVWA dashboard / application running

Screenshot 3 — Nmap service scan

Screenshot 4 — Nikto scan results

Screenshot 5 — Burp Suite intercepted request

Screenshot 6 — Burp Repeater request/response

Screenshot 7 — SQL Injection input

Screenshot 8 — SQL Injection result

Screenshot 9 — XSS input

Screenshot 10 — XSS result

Screenshot 11 — Final vulnerability findings

25. Conclusion

The project demonstrated a complete, controlled web application security testing workflow against an intentionally vulnerable DVWA environment. Nmap supported service discovery, Nikto assisted with web-server assessment, and Burp Suite enabled HTTP request and response analysis. SQL Injection and XSS demonstrations provided practical understanding of common application-layer vulnerabilities. The assessment highlights the importance of prepared statements, output encoding, secure configuration, security headers, least privilege, and safe error handling.

26. Project Deliverables

- Security assessment report
- Nmap scan output
- Nikto scan output
- Burp Suite evidence
- SQL Injection evidence
- XSS evidence
- Vulnerability findings and risk assessment
- Mitigation recommendations
- Screenshot evidence
- GitHub project documentation

27. Ethical and Safety Statement

All security testing described in this report is intended for authorized laboratory environments. The techniques should not be applied to systems, applications, accounts, networks, or data without explicit permission. The vulnerable application should remain isolated from production and public networks.

28. References

- OWASP Web Security Testing Guide (WSTG)
- OWASP Top 10 Web Application Security Risks
- PortSwigger Web Security Academy / Burp Suite documentation
- Nmap documentation
- Nikto documentation
- DVWA project documentation